Project Report
on
PLANT DISEASE DETECTION

project Report submitted in partial fulfillment of the requirement for the

award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

BINGIMALLA PRANATHI[R200356]

Under the Esteemed Guidance of

Mr. R.Sreenivasulu
Assistant Professor



RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP IIIT)

R.K Valley, Vempalli, Kadapa(Dist) – 516 330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-26

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP IIIT)

R.K.Valley,Vempalli(M),Kadapa(Dist)-516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-26

CERTIFICATE

This is to certify that the project report entitled**"PLANT DISEASE DETECTION"**is being submitted by **BINGIMALLA PRANATHI** under my guidance and supervision and is submitted to the **DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING** in partial fulfillment of the requirements for the award of the **Bachelor of Technology** in **Computer Science and Engineering** during the academic year **2025–26**, and it has been found worthy of acceptance according to the requirements of the University.

Signature of Internal Guide
Mr. R.Sreenivasulu
Assistant Professor
Department of CSE

Signature of HOD
Dr. Ch. RatnaKumari
Assistant Professor
Department of CSE

Signature of External Guide

ACKNOWLEGDEMENT

I wish to express our sincere thanks to various personalities who were responsible for the successful completion of the main project

I am grateful to **Dr. CH. RATNA KUMARI**, Head of the Department, for her motivation and encouragement in completing the project in specified time.

I express my deepfelt gratitude to **Mr**. **R. SREENIVASULU,** internal guide for his valuable guidance and encouragement which enabled me to successful completion of project in time.

I express my sincere thanks to all other faculty members of CSE Department for extending their helping hands and valuable suggestion when in need.

Finally, my heartfelt thanks to my parents for giving me all I ever needed to be a successful student and individual. Because of their hard work and dedication, I have had opportunities beyond my wildest dreams.

WITH SINCERE REGARDS

Bingimalla Pranathi
R200356

DECLARATION

Hereby declare that the project work entitled **“PLANT DISEASE DETECTION”** submitted to the **DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING** is a genuine work carried out by me for the fulfillment of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering during the Academic year 2025-26 under the supervision of our Project guide **Mr. R.Sreenivasulu Assistant Professor,Department of Computer Science and Engineering in RAJIV GANDHI UNIVERSITY OF KNOWLEDGE AND TECHNOLOGIES(AP IIIT),RK Valley**.

WITH SINCERE REGARDS

Bingimalla Pranathi

ABSTRACT

Agriculture plays a vital role in sustaining the global population, and plant health is a critical factor in ensuring high crop yield. Early and accurate detection of plant diseases is essential to prevent crop loss and maintain food security. Traditional disease detection methods are often time-consuming, require expert knowledge, and may not be feasible for large-scale monitoring.

This project presents an automated **Plant Disease Detection System** using Deep Learning techniques, specifically Convolutional Neural Networks (CNNs), to identify and classify diseases in plant leaves. The system allows users to upload images of leaves, preprocesses them, and predicts the corresponding disease class with high accuracy. To enhance interpretability, the project incorporates Grad-CAM visualizations, highlighting the regions of the leaf that most influenced the model's prediction.

The proposed system is implemented as a web application using Django, providing an intuitive interface for farmers, agricultural enthusiasts. Users can view predictions, visualize heatmaps, and maintain a history of analyzed images. Experimental results demonstrate that the model effectively distinguishes between healthy and diseased leaves across multiple plant species, offering a reliable tool for early disease detection and improved crop management.

Keywords: Plant Disease Detection, Deep Learning, CNN, Grad-CAM, Django, Image Classification, Agriculture

TABLE OF CONTENTS

TITLE 1

CERTIFICATE 2

ACKNOWLEDGEMENT 3

DECLARATION 4

ABSTRACT 5

CHAPTER 1: INTRODUCTION 8-11

- 1.1 Introduction
- 1.2 Background
- 1.3 Problem Statement
- 1.4 Objectives
- 1.5 Significance of Study
- 1.6 Scope and Limitations

CHAPTER 2: LITERATURE SURVEY 11–15

- 2.1 Literature Survey
- 2.2 Plant Disease Detection Techniques
- 2.3 Deep Learning in Agriculture
- 2.4 Image-based Disease Classification
- 2.5 Key Findings and Research Gaps
- 2.6 Conclusion

CHAPTER 3: SYSTEM OVERVIEW 16-18

CHAPTER 4: MODULE DESCRIPTION 19-22

- 4.1 User Registration and Login
- 4.2 Image Upload for Disease Detection
- 4.3 Disease Prediction Module (CNN)
- 4.4 Grad-CAM Visualization
- 4.5 Display Results and Suggestions
- 4.6 History of Uploaded Images

CHAPTER 5: MACHINE LEARNING MODEL 22-26

5.1 CNN Architecture (ResNet9)

5.2 Dataset and Preprocessing

5.3 Training and Validation

5.4 Model Integration with Django

CHAPTER 6: IMPLEMENTATION 27-32

6.1 Tools and Technologies Used

6.2 Overview of Views, Templates, and Filters

6.3 Image Upload, Prediction, and Display Workflow

CHAPTER 7: USER INTERFACE AND DASHBOARD 32-34

7.1 Upload Page and Result Display

7.2 Grad-CAM Heatmap Visualization

7.3 History and Analysis of Previous Uploads

CHAPTER 8: TESTING AND EVALUATION 35-36

8.1 Test Cases for Modules

8.2 Evaluation of CNN Model

CHAPTER 9: IMPLEMENTATION RESULTS 36-39

CHAPTER 10: FUTURE ENHANCEMENTS AND CONCLUSION 40

CHAPTER 11: REFERENCES 40

CHAPTER 1:

1.1 Introduction

Agriculture is the backbone of many economies, and crop health plays a crucial role in ensuring food security and economic stability. However, plant diseases significantly reduce agricultural productivity each year, leading to economic losses and food shortages. Traditional methods of disease detection rely on manual observation by farmers or agricultural experts, which are time-consuming, subjective, and often inaccurate. With the rapid advancements in Artificial Intelligence (AI) and Deep Learning, computer vision–based systems can now automatically detect plant diseases from leaf images with high accuracy. This project —Plant Disease Detection — aims to create an intelligent web-based system capable of identifying plant leaf diseases by analyzing images, thereby assisting farmers and researchers in early diagnosis and timely intervention.

1.2 Background

The agricultural sector forms the backbone of many economies, particularly in developing countries. However, the productivity of crops is often affected by various diseases caused by bacteria, fungi, or viruses. Early and accurate disease detection is essential for effective management and prevention.

Machine learning and deep learning have revolutionized this field by enabling automated, image-based plant disease identification. Convolutional Neural Networks (CNNs), a class of deep learning models, have shown exceptional performance in image classification tasks, making them ideal for detecting plant diseases from leaf images.In this project, the trained CNN model can recognize multiple crop types and their corresponding diseases. It has been developed using Python**,**

PyTorch, and Django, ensuring both robustness in prediction and user-friendliness in web interaction.

1.3 Problem Statement

Traditional plant disease detection methods depend on expert knowledge, laboratory tests, or manual visual inspection — all of which are time-consuming, inconsistent, and inaccessible to small-scale farmers. The lack of immediate and affordable detection methods results in the late identification of diseases, leading to significant yield loss.

Hence, there is a need for an automated, accurate, and easily accessible plant disease detection system that can help farmers and agricultural researchers identify diseases at an early stage through a simple image upload.

1.4 Objectives

The main objectives of this project are:

1. To develop a Convolutional Neural Network (CNN) model capable of accurately classifying plant leaf diseases.
2. To integrate the model into a Django-based web interface for easy image upload and real-time prediction.
3. To display the predicted disease label along with its confidence score for better interpretability.
4. To maintain an upload history for users, showing their past predictions and results.
5. To create a dashboard that visualizes data such as number of uploads, plant distribution, and model performance metrics.
6. To assist in early disease diagnosis and improve agricultural productivity through digital tools.

1.5 Significance of Study

This project demonstrates how artificial intelligence can be applied effectively in agriculture, addressing one of the most crucial challenges —disease detection. The model reduces the dependency on expert analysis, saves time, and provides instant results.

By deploying it as a web application, this system becomes accessible to anyone with an internet connection, including farmers, agronomists, and researchers. It supports sustainable farming practices and helps in early disease management, ultimately improving crop yield and reducing losses.

1.6 Scope and Limitations

Scope:

The project covers the detection of multiple plant diseases using images of infected leaves. The model can identify diseases across several plant species, including:

- **Apple:** Cedar Apple Rust, Black Rot, Apple Scab, Healthy
- **Grape:** Leaf Blight (Isariopsis Leaf Spot), Black Rot, Esca (Black Measles), Healthy
- **Tomato:** Yellow Leaf Curl Virus, Target Spot, Septoria Leaf Spot, Spider Mites, Early Blight, Late Blight, Bacterial Spot, Mosaic Virus, Leaf Mold, Healthy
- **Corn (Maize):** Common Rust, Northern Leaf Blight, Cercospora Leaf Spot (Gray Leaf Spot), Healthy
- **Potato:** Late Blight, Early Blight, Healthy
- **Peach:** Bacterial Spot, Healthy
- **Pepper (Bell):** Bacterial Spot, Healthy

- **Cherry (including sour):** Powdery Mildew, Healthy
- **Strawberry:** Leaf Scorch, Healthy
- **Orange:** Haunglongbing (Citrus Greening)
- **Blueberry, Raspberry, Soybean, Squash:** Healthy

The system is designed to classify these diseases using images uploaded by users and can be further extended to include more plant types and diseases as data becomes available.

Limitations:

- The accuracy depends on image quality and lighting conditions.
- The model may not generalize well for unseen plant species or disease types not present in the training dataset.
- Internet access is required to use the web application.
- Field-level real-time detection may require further optimization and mobile deployment.

CHAPTER 2: LITERATURE SURVEY

2.1 Literature Survey

Plant diseases are one of the major causes of crop yield reduction and economic loss across the globe. Farmers often rely on visual inspection or expert consultation to identify plant diseases, which is time-consuming, expensive, and sometimes inaccurate. Over the past decade, researchers have explored various computational approaches to automate the process of disease detection using image processing and machine learning.

Early studies primarily used traditional image processing techniques involving color, texture, and shape features extracted from leaf images. These handcrafted features were then classified using algorithms such

as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), or Decision Trees. However, the accuracy of such methods was often limited due to the complex nature of plant diseases and varying environmental conditions.

With advancements in Deep Learning (DL), particularly Convolutional Neural Networks (CNNs), the accuracy and robustness of plant disease detection have improved significantly. CNNs can automatically extract hierarchical features from raw images, removing the need for manual feature engineering and improving classification performance even under complex conditions such as varying light, orientation, or background noise.

2.2 Plant Disease Detection Techniques

Plant disease detection techniques can be broadly categorized into three types:

1. **Manual Detection Techniques:**

   In traditional agricultural practice, plant diseases are diagnosed visually by experts or agricultural officers based on symptoms such as color changes, spots, or leaf deformations. While effective in small-scale farms, this method is inefficient for large-scale cultivation and is subject to human bias.

2. **Image Processing-based Techniques:**

   In these methods, leaf images are captured and processed using techniques like segmentation, edge detection, and feature extraction. Algorithms such as Gray-Level Co-occurrence Matrix (GLCM) for texture analysis or Color Histograms for color variation were commonly used to identify disease patterns. Classification was performed using SVM, Random Forest, or KNN algorithms.

However, the accuracy was often limited when the image background or lighting conditions varied.

3. **Deep Learning-based Techniques:**

   Deep Learning has revolutionized plant disease identification by using neural networks that automatically learn complex feature representations. CNN architectures such as VGG16, ResNet50, and InceptionV3 have been successfully applied for disease classification tasks. These models have achieved accuracies exceeding 95% on benchmark datasets like PlantVillage, demonstrating their superiority over traditional approaches.

2.3 Deep Learning in Agriculture

Deep learning models have found extensive applications in agriculture beyond plant disease detection. They are used in tasks such as weed detection, yield prediction, soil analysis, and crop type classification. The ability of CNNs to learn from large amounts of image data makes them ideal for tasks that involve pattern recognition and visual interpretation.

In plant pathology, CNNs can distinguish between healthy and diseased leaves even when visual differences are subtle. Researchers have experimented with pre-trained models through Transfer Learning, which reduces the need for massive labeled datasets. Using pre-trained models like MobileNet or EfficientNet allows rapid deployment on devices with limited computational resources, including smartphones — making the technology practical for farmers.

Furthermore, integrating these models into web or mobile applications using frameworks like Django or Flask enables real-time disease detection accessible to a wide audience.

2.4 Image-based Disease Classification

Image-based classification forms the foundation of this project. The process involves four major stages:

1. Image Acquisition:

   Leaf images are captured using mobile phones or cameras under various environmental conditions.

2. Preprocessing:
   Images are resized, normalized, and augmented to improve the model's generalization. Techniques like flipping, rotation, and contrast adjustment are used to create a diverse training dataset.

3. Feature Extraction and Model Training:
   A Convolutional Neural Network (CNN) automatically extracts spatial and color features. The CNN layers capture hierarchical representations, from basic patterns (edges, textures) in the early layers to complex disease features in deeper layers.

4. Prediction and Evaluation:

   The trained model predicts the disease category and outputs the confidence score. The model used in this project is trained on a dataset that includes 14 different plant species such as Tomato, Apple, Corn, Grape, Potato, Peach, Orange, Strawberry, and others, covering diseases like Early Blight, Late Blight, Leaf Mold, Black Rot, Powdery Mildew, and Citrus Greening.

By using CNNs, the proposed system minimizes the error rate and achieves high detection accuracy even for multiple disease classes under variable image conditions.

2.5 Key Findings and Research Gaps

From the review of existing literature, the following key findings and research gaps were identified:

Key Findings:

- Deep learning models, particularly CNNs, have achieved outstanding results in image-based plant disease detection.
- Transfer learning significantly reduces training time and improves accuracy with limited datasets.
- Integrating detection systems into user-friendly applications enhances accessibility for farmers.
- Image augmentation and preprocessing techniques play a vital role in improving model performance.

Research Gaps:

- Many existing models are trained on controlled datasets (like PlantVillage) and may not perform equally well on real-world images with varying lighting, occlusions, or backgrounds.
- Most models focus only on classification, not on providing actionable insights such as disease severity or treatment recommendations.
- Few systems provide an integrated platform combining prediction history, data visualization, and user analytics.
- Deployment of deep learning models on lightweight or mobile devices remains a technical challenge due to high computational requirements.

2.6 Conclusion

The literature review demonstrates the rapid evolution of plant disease detection technologies — from manual identification to deep learning-driven automation. Among the various techniques, Convolutional Neural Networks (CNNs) have proven to be the most effective for accurate classification and diagnosis.

CHAPTER 3: SYSTEM OVERVIEW

3.1 System Overview

The Plant Disease Detection System is a deep learning–based web application that enables automatic identification of plant diseases from leaf images. The system uses a Convolutional Neural Network (CNN) trained on a dataset containing multiple plant species and disease categories.
The main goal of this project is to assist farmers and researchers in identifying plant diseases early, thereby preventing crop loss and ensuring better yield quality.

This system allows users to upload an image of a plant leaf through an intuitive web interface. The trained deep learning model processes the image and predicts the corresponding disease along with a confidence score. The system also visualizes the decision-making process of the CNN using Grad-CAM heatmaps, highlighting the regions of the leaf that contributed most to the model's prediction.

3.2 System Architecture

The system follows a three-tier architecture consisting of the following layers:

1. Presentation Layer (Frontend):

   This layer provides the user interface for uploading images, viewing predictions, and exploring Grad-CAM visualizations. It is designed using HTML, CSS, and JavaScript, ensuring a clean, responsive, and user-friendly experience.

2. Application Layer (Backend):

   The backend is implemented using Django, a high-level Python web framework. It manages user authentication, image uploads,

prediction requests, and result storage. Django also serves as the bridge between the trained CNN model and the user interface.

3. Data Layer (Model and Database):

   The trained CNN model (implemented in PyTorch) is responsible for predicting plant diseases based on the uploaded image. The results, including the predicted class, confidence score, and timestamp, are stored in the database (SQLite).

3.3 System Workflow

The working of the system can be described in the following sequential steps:

1. User Registration and Login:
   The user creates an account and logs into the system to access features like image uploads and viewing history.

2. Image Upload:
   The user uploads a clear image of a plant leaf. The system validates the file format and stores it in the media directory.

3. Model Prediction:
   The uploaded image is passed to the CNN model, which performs feature extraction and classification.

   - The CNN analyzes patterns such as spots, discoloration, texture variations, and edges on the leaf.
   - It then predicts the plant species and type of disease along with a confidence score.

4. Grad-CAM Heatmap Generation:
   After prediction, a Gradient-weighted Class Activation Mapping (Grad-CAM) technique is applied to the model's last convolutional layer. This generates a heatmap overlay on the input image, showing the regions that influenced the model's prediction.

5. Result Visualization:
   The system displays the original leaf image, predicted disease name, confidence percentage, and Grad-CAM heatmap on the results page. Users can toggle between the original and heatmap view.

6. Upload History and Dashboard:
   The application also maintains a history of all predictions for each user. A dashboard provides statistics such as:

   - Number of images analyzed
   - Most frequently detected diseases
   - Model confidence distribution over time

3.4 Components of the System

Component	Description
Frontend (UI)	Developed using HTML, CSS, and JavaScript. Provides responsive and visually appealing pages for upload, results, and dashboard.
Backend (Django)	Handles routing, database operations, model loading, and business logic.
CNN Model (PyTorch)	Pre-trained deep learning model that classifies plant diseases into one of the defined categories.
Grad-CAM Module	Generates visual explanations by overlaying heatmaps to show the important regions used for prediction.
Database (SQLite)	Stores user details, upload history, predicted results, and timestamps.
Static & Media Files	Manages CSS, JS, and uploaded images for smooth frontend–backend integration.

CHAPTER 4: MODULE DESCRIPTION

4.1 User Registration and Login

The User Registration and Login module provides secure access to the system and maintains individual user data.
Each user must register by providing details such as username, password, email, and phone number. Django's authentication framework ensures password encryption and secure session management.

Once registered, users can log in to the system to access functionalities like uploading images, viewing predictions, and tracking upload history. The login module validates user credentials against the database before granting access. Session-based authentication prevents unauthorized access, and users can log out securely after use.

Features:

- User authentication using Django's built-in model
- Form validation and error handling
- Secure password hashing
- Session-based access control

4.2 Image Upload for Disease Detection

This module allows users to upload an image of a plant leaf through an intuitive web interface.
The uploaded image serves as the input to the CNN model for disease detection. Before saving, the system verifies the image format (JPEG/PNG) and stores it in the media directory for processing.

Once the image is uploaded successfully, the path of the image is passed to the disease prediction module. The image upload interface is designed to be responsive and simple, ensuring ease of use even for non-technical users such as farmers.

Key Functionalities:

- Accepts image files in .jpg, .jpeg, or .png format
- Automatically stores images in the media folder
- Displays upload confirmation before prediction
- Stores metadata such as upload time and user ID in the database

4.3 Disease Prediction Module (CNN)

The Disease Prediction Module is the core component of the system, powered by a Convolutional Neural Network (CNN) built using PyTorch. The model was trained on a labeled dataset containing 14 plant species and 38 disease classes, including both healthy and infected samples.

When a new image is uploaded, it undergoes preprocessing steps such as resizing, normalization, and tensor conversion. The CNN model extracts deep visual features — like spots, discolorations, and texture variations — to classify the image into one of the predefined categories.

The system then outputs:

- Predicted Plant Name (e.g., Tomato, Apple, Grape)
- Predicted Disease Type (e.g., Late Blight, Black Rot, Powdery Mildew)
- Confidence Score (probability percentage of prediction accuracy)

Model Details:

- Architecture: ResNet9 (custom CNN architecture)
- Framework: PyTorch
- Activation Function: ReLU
- Pooling: Max Pooling for spatial reduction
- Optimization: Adam Optimizer

- Loss Function: Cross-Entropy Loss

4.4 Grad-CAM Visualization

To enhance transparency and interpretability, the system incorporates Grad-CAM (Gradient-weighted Class Activation Mapping).
This visualization technique highlights the regions of the leaf that contributed most to the CNN's prediction.

After the model predicts the class, Grad-CAM extracts gradients from the final convolutional layer and generates a heatmap overlay on the original image.
The red and yellow areas in the heatmap represent the regions the model focused on while identifying the disease.

The generated heatmap is stored in the static folder and displayed to the user on the Model Layout (Grad-CAM) page. Users can toggle between the original image and the heatmap overlay for better understanding.

Benefits:

- Provides model interpretability
- Helps in identifying which parts of the leaf indicate disease
- Enhances trust and reliability in AI predictions

4.5 Display Results and Suggestions

After the prediction and Grad-CAM generation, the system displays the results in a user-friendly manner.
The result page shows:

- Original Uploaded Image
- Predicted Plant and Disease Name
- Confidence Percentage
- Grad-CAM Heatmap Visualization

4.6 History of Uploaded Images

The Upload History module allows users to view all the images they have uploaded, along with their corresponding prediction details. Each record contains:

- Image thumbnail
- Disease name and plant type
- Confidence score
- Upload date and time

This module is connected to the UploadHistory model in Django, which stores each prediction's metadata in the database. Users can revisit old predictions, download images, and analyze model performance over time. This feature also helps in tracking disease progression on particular crops.

Functionalities:

- Display all previous uploads in a structured grid view
- Search or filter uploads by plant name or date
- Retrieve prediction details instantly

CHAPTER 5: MACHINE LEARNING MODEL

5.1 CNN Architecture (ResNet9)

The core of the plant disease detection system is a Convolutional Neural Network (CNN) designed to automatically extract image features and classify diseases in plants. The architecture used in this project is based on ResNet9, a lightweight yet efficient model that utilizes residual connections to improve gradient flow and prevent vanishing gradients.

The network consists of multiple convolutional layers, batch normalization, and ReLU activation functions followed by max-pooling

layers. The residual blocks enable the model to learn complex feature hierarchies efficiently. Finally, fully connected layers map the learned features to output classes representing various plant diseases.

This architecture provides an optimal trade-off between accuracy and computational efficiency, making it suitable for deployment in a web application environment.

Resnet9 (that I have used in model):

```
import torch

import torch.nn as nn

class ResNet9(nn.Module):

    def __init__(self,in_channels,num_classes):

        super().__init__()

        self.conv1=nn.Sequential(

            nn.Conv2D(in_channels,64,3,1,1),

            nn.BatchNorm2D(64),

            nn.ReLU(),

            nn.MaxPool2D(2,2)

        )

        self.conv2=nn.Sequential(

            nn.Conv2D(in_channels,64,3,1,1),

            nn.BatchNorm2D(64),

            nn.ReLU(),

            nn.MaxPool2D(2,2)

        )

        self.conv3=nn.Sequential(
```

```
            nn.Conv2D(in_channels,64,3,1,1),
            nn.BatchNorm2D(64),
            nn.ReLU(),
            nn.MaxPool2D(2,2)
        )
        self.res2=nn.Sequential(
            nn.Conv2D(256,256,3,1,1),
            nn.BatchNorm2D(256),
            nn.ReLU(),
        )
        self.classifier=nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(256,num_classes)
        )
    def  forward(self,xb):
        out=self.conv1(xb)
        out=self.conv2(out)
        out=out+self.res1(out)
        out=self.conv3(out)
        out=out+self.res1(out)
        out=self.classifier(out)
        return out
```

5.2 Dataset and Preprocessing

The dataset used for training was derived from the PlantVillage dataset, which contains over 70,000 labeled images of healthy and diseased plant leaves. For this project, the dataset focuses on the following plant species and their diseases:

- **Apple:** Cedar Apple Rust, Black Rot, Apple Scab, Healthy
- **Grape:** Leaf Blight (Isariopsis Leaf Spot), Black Rot, Esca (Black Measles), Healthy
- **Tomato:** Yellow Leaf Curl Virus, Target Spot, Septoria Leaf Spot, Spider Mites, Early Blight, Late Blight, Bacterial Spot, Mosaic Virus, Leaf Mold, Healthy
- **Corn (Maize):** Common Rust, Northern Leaf Blight, Cercospora Leaf Spot (Gray Leaf Spot), Healthy
- **Potato:** Late Blight, Early Blight, Healthy
- **Peach:** Bacterial Spot, Healthy
- **Pepper (Bell):** Bacterial Spot, Healthy
- **Cherry (including sour):** Powdery Mildew, Healthy
- **Strawberry:** Leaf Scorch, Healthy
- **Orange:** Haunglongbing (Citrus Greening)
- **Blueberry, Raspberry, Soybean, Squash:** Healthy

Each image was resized to a standard dimension of 256×256 pixels and normalized to improve model convergence.
Data augmentation techniques such as rotation, flipping, zooming, and brightness adjustment were applied to increase dataset diversity and reduce overfitting.

5.3 Training and Validation

The model was trained using PyTorch with cross-entropy loss as the objective function and the Adam optimizer for adaptive learning rate adjustment.

- Batch size: 32
- Learning rate: 0.001
- Epochs: 5
- Validation split: valid folder (separate)

During training, the model's performance was continuously evaluated on a validation set to monitor convergence and prevent overfitting using early stopping and learning rate scheduling.

5.4 Model Integration with Django

The trained CNN model was exported as a .pth file using PyTorch and integrated into the Django web framework. When a user uploads an image of a plant leaf, the system:

1. Loads the pre-trained model.
2. Preprocesses the uploaded image (resize, normalize, and tensor conversion).
3. Performs inference using the CNN model.
4. Returns the predicted disease label and confidence score.

The results are then displayed on the web interface, along with Grad-CAM visualizations for better interpretability.
This seamless integration allows users to interact with the trained AI model through a user-friendly web application, enabling real-time plant disease diagnosis.

CHAPTER 6: IMPLEMENTATION

6.1 Tools and Technologies Used

The plant disease detection system combines machine learning, computer vision, and web development technologies to provide a seamless end-to-end solution. The following tools and technologies were used during implementation:

Backend

- Django Framework (Python): Used for developing the backend logic, managing user authentication, routing, and database interactions.
- PyTorch: Used to train and deploy the Convolutional Neural Network (CNN) model for disease prediction.
- SQLite3: Serves as the project's database for storing user profiles, uploaded images, predictions, and history.

Frontend

- HTML5, CSS3, JavaScript: Used to create responsive and visually appealing web pages for user interaction.
- Bootstrap: Provides a responsive layout and UI components for better design consistency.

Machine Learning / Deep Learning

- NumPy, Pandas: For data manipulation and preprocessing.
- Torchvision: For dataset handling, transformations, and model utilities.
- Matplotlib**:** For visualizing training metrics and confusion matrices.

Other Tools

- VS Code: Development environments used for coding and debugging.
- Google Colab: Used for model training using GPU acceleration.

6.2 Overview of Views, Templates, and Filters

The Django framework separates the project into Model–View–Template (MVT) components for better structure and maintainability.

Views

Views handle the core logic of the application. Key views include:

- home – Displays the homepage with information about the system.
- register and login – Manage user registration and authentication.
- upload_image – Handles image upload and sends the image to the prediction model.
- predict_disease – Loads the trained CNN model and predicts the plant disease.
- history – Displays a list of previously uploaded images and their prediction results.
- dashboard – Shows aggregated statistics such as the number of uploads and common diseases detected.

```
# detector/views.py

#  Upload Image

@login_required

def upload_image(request):

    if request.method == "POST" and request.FILES.get("image"):

        uploaded_file = request.FILES["image"]
```

```
        # Save uploaded image
        save_path = os.path.join(settings.MEDIA_ROOT, uploaded_file.name)
        with open(save_path, 'wb+') as f:
            for chunk in uploaded_file.chunks():
                f.write(chunk)
        # Predict
        try:
            predicted_class, confidence = predict_image(save_path)
            print(f"[DEBUG] Predicted: {predicted_class}, Confidence: {confidence:.2f}%")
        except Exception as e:
            predicted_class = "Unknown"
            confidence = 0
            print(f"Prediction error: {e}")


        # Extract plant name
                plant_name = predicted_class.split('___')[0] if '___' in predicted_class else
predicted_class.split('_')[0]
        # Save prediction in DB
        prediction = Prediction.objects.create(
            user=request.user,
            image=uploaded_file,
            predicted_label=predicted_class,
            confidence=confidence
        )
```

```
        return redirect('result_view', pk=prediction.pk)
    return render(request, "detector/upload.html")
# Result View
@login_required
def result_view(request, pk):
    prediction = get_object_or_404(Prediction, pk=pk, user=request.user)
    plant_name = (
        prediction.predicted_label.split('___')[0]
        if '___' in prediction.predicted_label
        else prediction.predicted_label.split('_')[0]
    )
    request.session['image_url'] = prediction.image.url
    context = {
        'image_url': prediction.image.url,
        'predicted_class': prediction.predicted_label,
        'confidence': round(prediction.confidence, 2),
        'plant_name': plant_name,
        'timestamp': prediction.created_at if hasattr(prediction, 'created_at') else None,
    }
    return render(request, 'detector/result.html', context)
```

Templates are HTML files stored under the templates/ directory. Major templates include:

- home.html – Landing page describing the project and features.
- upload.html – Interface for uploading plant leaf images.

- result.html – Displays predicted disease label, confidence score, and Grad-CAM visualization.
- history.html – Displays previous predictions for each user.
- login.html and register.html – Authentication pages for users.

Each template uses Django template tags to dynamically display data fetched from views.

Filters

Filters are used to refine displayed data in templates. For example, dates are formatted using `{{ date|date:"d M Y" }}`, and prediction confidence values can be rounded using custom filters.

6.3 Image Upload, Prediction, and Display Workflow

The complete workflow from image upload to displaying prediction results is as follows:

1. **User Uploads Image:**
   The user logs into the system and uploads an image of a plant leaf through the upload.html page.
2. **Image Processing:**
   The uploaded image is saved in the MEDIA_ROOT/uploads/ directory. It is then preprocessed — resized, normalized, and converted into a tensor format compatible with the CNN model.
3. **Model Prediction:**
   The pre-trained ResNet9-based CNN model is loaded, and inference is performed on the processed image. The model outputs the predicted disease label (e.g., *Tomato Leaf Mold*) along with a confidence score.

4. **Result Display:**
   The prediction results are displayed on the result.html page along with:
   - Disease name
   - Confidence percentage
   - Grad-CAM visualization showing the affected leaf region

CHAPTER 7: USER INTERFACE AND DASHBOARD

7.1 Upload Page and Result Display

The Upload Page is the primary interaction point where users can submit plant leaf images for disease detection. The design focuses on simplicity, responsiveness, and clarity, ensuring that even users with limited technical knowledge can easily operate it.

Features:

- Users can upload an image directly from their device using the file input form.
- A preview of the uploaded image is displayed before submission.
- Upon clicking the “Predict Disease” button, the image is sent to the backend Django view, where it is processed and classified by the trained CNN model.
- The system predicts the disease name and provides a confidence score (in %) indicating the model’s certainty.

Result Display:

Once the prediction is complete, users are redirected to the Result Page, which provides:

- The predicted plant disease name (e.g., *Tomato Leaf Mold*, *Apple Black Rot*, *Healthy*).
- The confidence level of the prediction (e.g., *92.4% confidence*).
- The affected plant name automatically derived from the uploaded image category.
- A short description of the disease, its symptoms, and suggested treatments or preventive measures.

The result page is aesthetically designed with a card-based layout, using soft colors and readable typography for better user experience. This section bridges the gap between AI model outputs and user understanding by providing meaningful, interpretable results.

7.2 Grad-CAM Heatmap Visualization

To enhance interpretability, the system integrates Grad-CAM (Gradient-weighted Class Activation Mapping) — a deep learning visualization technique used to highlight the regions in the image that contributed most to the model's prediction.

Working Principle:

- Grad-CAM uses the gradients of the target class flowing into the final convolutional layer of the CNN to produce a localization heatmap.
- This heatmap highlights important regions on the plant leaf that influenced the model's decision.

User Display:

- The Grad-CAM output is displayed alongside the original uploaded image on the result page.
- The heatmap overlay clearly shows infected regions in reddish tones, while less significant areas remain in cooler shades.

- This visualization helps users and researchers understand why the model predicted a particular disease, improving trust and transparency in AI predictions.

7.3 History and Analysis of Previous Uploads

The system includes a History Page that maintains a record of all images uploaded by the user along with their corresponding predictions. This enables users to track disease trends and analyze plant health over time.

Features of the History Section:

- Displays a list/grid of previously uploaded images with details such as:
    - Upload date and time
    - Predicted disease label
    - Confidence score
- Users can click on any previous entry to re-view the prediction details and Grad-CAM visualization.

Data Analysis and Dashboard:

For administrators and researchers, a dashboard analytics page provides visual insights into overall system usage. It includes:

- Pie Chart: Showing distribution of diseases detected (e.g., Tomato Leaf Mold – 30%, Black Rot – 15%, Healthy – 25%, etc.)
- Line Chart: Displaying trends in predictions or confidence scores over time.

These charts are generated dynamically using Chart.js, offering an interactive and visually appealing way to interpret system data. This analytics component supports performance monitoring and helps in refining the model for better accuracy.

CHAPTER 8: TESTING AND EVALUATION

8.1 Test Cases for Modules

To ensure the reliability, functionality, and accuracy of the plant disease detection system, comprehensive testing was carried out across all modules. The primary goal of testing was to verify that each component performs as expected under normal and boundary conditions.

Module	Test Scenario	Input	Expected Output	Result
User Registration	Register with valid credentials	Username, Email, Password	Registration successful; redirected to login	Pass
User Registration	Register with existing email	Duplicate Email	Error message: “Email already exists”	Pass
Login Module	Login with valid credentials	Username, Password	Redirect to home/dashboard	Pass
Login Module	Login with invalid password	Username, Wrong Password	Display error: “Invalid credentials”	Pass
Image Upload	Upload a valid plant leaf image	`.jpg` or `.png` file	Image successfully uploaded; preview displayed	Pass
Image Upload	Upload invalid file type	`.txt` or `.pdf` file	Error message: “Invalid file format”	Pass
Prediction Module (CNN)	Predict using valid image	Leaf image	Displays predicted disease and confidence score	Pass
Grad-CAM Visualization	Generate heatmap	Uploaded image	Displays heatmap highlighting infected region	Pass
History Module	View previous uploads	User logged in	Displays list of previously uploaded images	Pass
Dashboard Analytics	View overall analysis	Admin access	Displays disease distribution charts	Pass

All modules performed successfully under various test cases, confirming that the system operates smoothly, with robust error handling and accurate feedback to users.

8.2 Evaluation of CNN Model

The Convolutional Neural Network (CNN) model used in this system was trained on the PlantVillage dataset, which contains over 70,000+ labeled images across multiple plant species and disease categories. The model was designed to identify and classify leaf diseases for plants including Tomato, Apple, Corn, Grape, Potato, Pepper, Orange, Strawberry, Cherry, Blueberry, Soybean, Squash, and Peach.

Dataset Split:

- Training Set: train data folder
- Validation Set: valid data folder
- Testing Set: test data folder

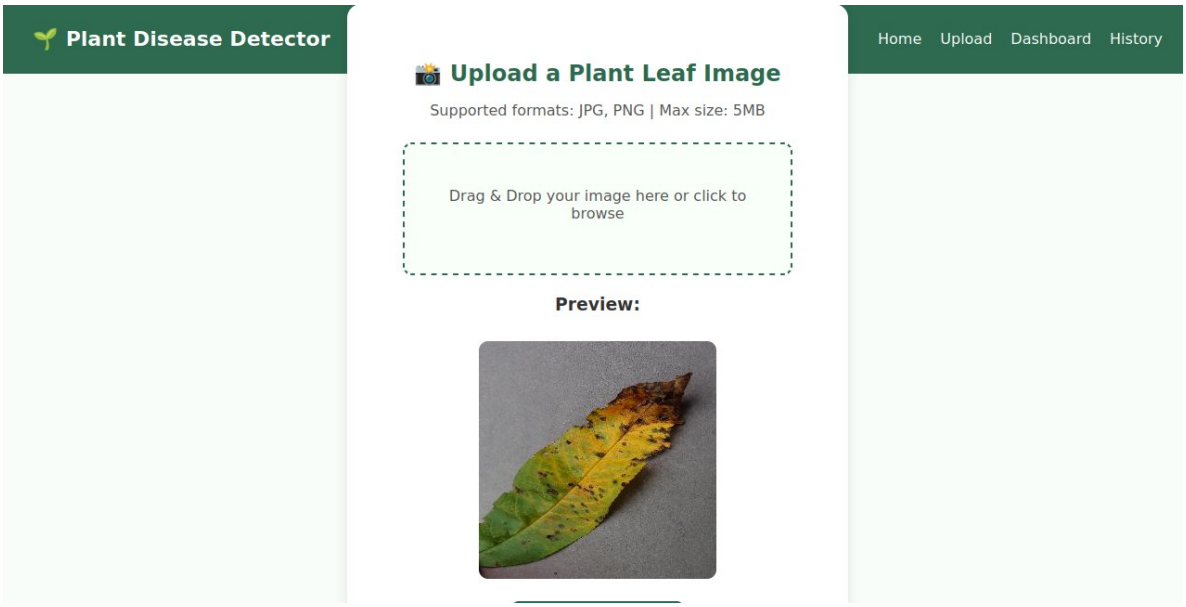
The model was trained using PyTorch, optimized using the Adam optimizer, and employed Cross-Entropy Loss as the objective function. Data augmentation techniques such as rotation, flipping, and normalization were used to enhance model generalization.

CHAPTER 9: IMPLEMENTATION RESULTS

Implementation of Home page



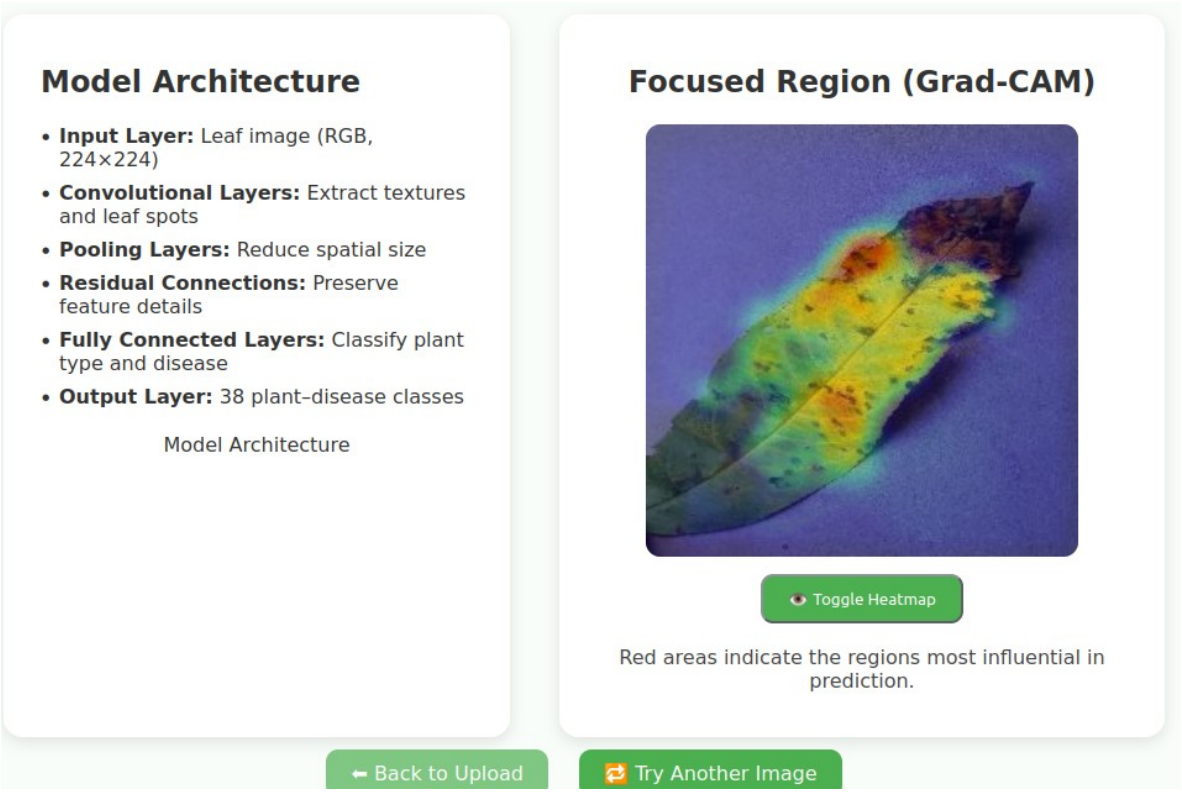
Implementation of Upload page



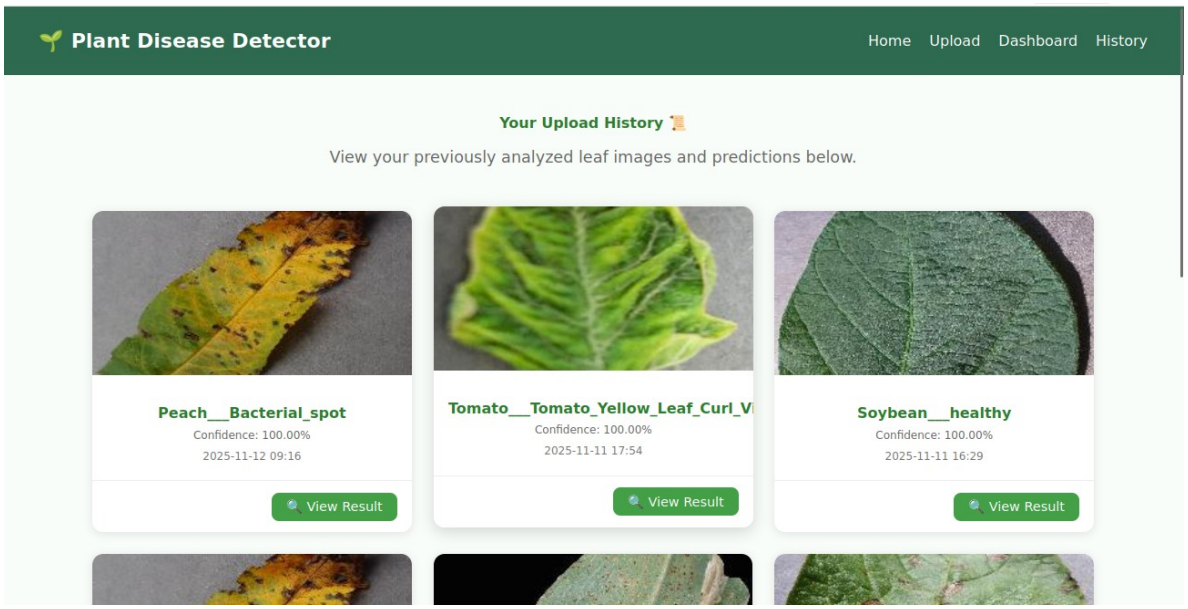
Implementation of Result Page



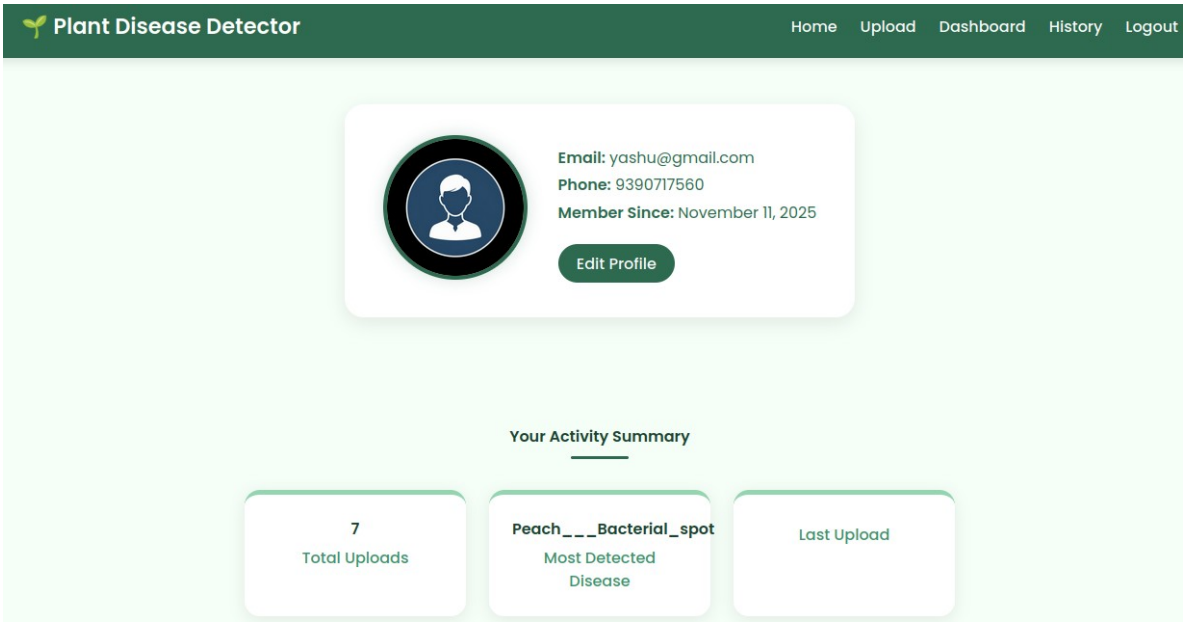
Implementation of Layout Page



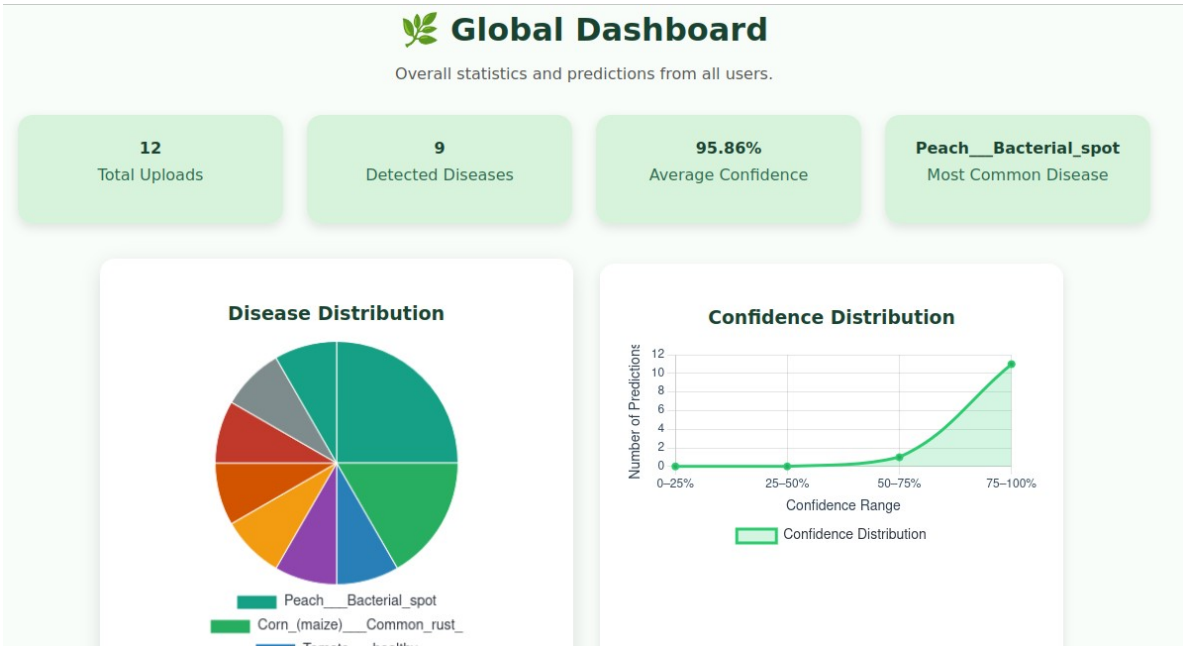
Implementation of History Page(For user)



Implementation of Profile Page



Implementation of Dashboard Page(for website)



CHAPTER 10: FUTURE ENHANCEMENTS AND CONCLUSION

The Plant Disease Detection System using CNN and Django demonstrates how deep learning can support agriculture by automating plant disease identification from leaf images. With an integrated CNN model, the system enables users to upload images, view predictions with confidence scores, and analyze affected regions using Grad-CAM. It effectively identifies diseases across multiple crops such as tomato, apple, grape, corn, and potato.

While the system performs well, future improvements can include training on larger datasets, enabling real-time field image capture. Additional features like weather-based predictions, treatment suggestions, and voice assistance can further enhance usability.

In conclusion, this project contributes to digital agriculture by providing a practical and intelligent tool that assists early disease detection, helping farmers improve crop health and productivity.

CHAPTER 11: REFERENCES

1. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. Frontiers in Plant Science, 7, 1419.
2. Ferentinos, K. P. (2018). Deep Learning Models for Plant Disease Detection and Diagnosis. Computers and Electronics in Agriculture, 145, 311–318.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition (ResNet). Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.
4. Kaggle. *Plant Village Dataset for Plant Disease Classification.* https://www.kaggle.com